TOPIC 18.0: MATH FOUNDATION & OVERFLOW PROTECTION

🔗 See Topic 17 → Bit Manipulation Masterclass

🎯 **GOAL:** Master Interview Mathematics! Learn Euclidean GCD/LCM, Sieve of Eratosthenes, Binary Exponentiation `Pow(x, n)`, Modular Arithmetic, Cross Product Slope Checks, and solve all NeetCode Math questions!

🧰 REUSABLE MATHEMATICAL UTILITIES

GCD & LCM (Euclidean Algorithm):

```
long gcd(long a, long b) { return b == 0 ? a : gcd(b, a % b); }
long lcm(long a, long b) { return (a / gcd(a, b)) * b; }
```

Binary Exponentiation O(log N):

```
long power(long base, long exp, long mod) {
    long res = 1; base %= mod;
    while (exp > 0) {
        if ((exp & 1) == 1) res = (res * base) % mod;
        base = (base * base) % mod; exp >>= 1;
    }
    return res;
}
```

💡 **AHA MOMENT #1: PREVENTING INTEGER OVERFLOW IN INTERVIEWS**

Java 32-bit `int` overflows at $2^{31}-1 = 2,147,483,647$.

- **Multiplication Overflow:** `(long) a * b` forces 64-bit calculation before assigning back or taking modulo!
- **LCM Overflow:** Compute `(a / gcd(a, b)) * b` (divide FIRST!) instead of `(a * b) / gcd(a, b)`!
- **Floating Point Precision:** Never use `==` on `double`. Use `Math.abs(a - b) < 1e-9`!

1 3 OVERFLOW PROTECTION RULES

1. Cast to `long` before multiplying: `long prod = (long) a * b;`
2. Divide before multiply in LCM: `(a / gcd) * b`
3. Use Modulo subtraction safety: `(a - b + M) % M`

2 FLOATING POINT EPSILON COMPARISON

In Java, `0.1 + 0.2 == 0.3` evaluates to `false` due to binary IEEE-754 representation! Always compare using Epsilon ($\epsilon = 10^{-9}$):

`Math.abs(val1 - val2) < 1e-9`

📌 **KEY TAKEAWAY:** Always check if intermediate calculations can exceed 2×10^9 and cast to `long`!

∞ TOPIC 18.1: GCD & LCM (EUCLIDEAN ALGORITHM)

💡 **AHA MOMENT #2: WHY EUCLIDEAN GCD WORKS IN $O(\log(\min(a, b)))$ TIME**
Mathematical Principle: $\gcd(a, b) = \gcd(b, a \bmod b)$.
Any divisor that divides both a and b MUST also divide $a - k \cdot b = a \bmod b$.
Because $a \bmod b < a / 2$, the numbers decrease by at least half every two steps, achieving logarithmic $O(\log(\min(a, b)))$ convergence!

1 RECURSIVE & ITERATIVE GCD / LCM

```
public static long gcd(long a, long b) {
    return b == 0 ? a : gcd(b, a % b);
}
public static long lcm(long a, long b) {
    if (a == 0 || b == 0) return 0;
    return (a / gcd(a, b)) * b; // Divide FIRST to prevent overflow!
}
```

2 EUCLIDEAN TRACE (A = 48, B = 18)

```
Step 1: gcd(48, 18) ➔ 48 % 18 = 12 ➔ gcd(18, 12) Step 2: gcd(18, 12) ➔
18 % 12 = 6 ➔ gcd(12, 6) Step 3: gcd(12, 6) ➔ 12 % 6 = 0 ➔ gcd(6, 0)
Step 4: b == 0 ➔ Return a = 6! (GCD is 6)
```

📌 **KEY TAKEAWAY:** `gcd(a, b) * lcm(a, b) = a * b`. Always divide by GCD before multiplying to compute LCM!

📐 TOPIC 18.2: PRIME NUMBERS & SIEVE OF ERATOSTHENES

💡 **AHA MOMENT #3: SIEVE OF ERATOSTHENES IN $O(N \log \log N)$ TIME**

To count primes up to N:
1. Create `boolean[] isPrime` array initialized to `true`.
2. For each prime $p \le \sqrt{N}$, mark all its multiples $p^2, p^2 + p, p^2 + 2p \dots$ as `false`!
Starting marking at p^2 skips already marked multiples from smaller primes!

🛠️ SIEVE OF ERATOSTHENES (LC 204 COUNT PRIMES)

```
public int countPrimes(int n) {
    if (n ⩽ 2) return 0;
    boolean[] isPrime = new boolean[n];
    Arrays.fill(isPrime, true);
    isPrime[0] = isPrime[1] = false;
    for (int p = 2; p * p < n; p++) {
        if (isPrime[p]) {
            for (int i = p * p; i < n; i += p) { // Start at p * p!
                isPrime[i] = false;
            }
        }
    }
    int count = 0;
    for (boolean b : isPrime) if (b) count++;
    return count;
}
```

📌 **KEY TAKEAWAY:** Sieve reduces prime generation from $O(N \sqrt{N})$ to $O(N \log \log N) \approx O(N)$ linear time!

⚖️ TOPIC 18.3: MODULAR ARITHMETIC LAWS

💡 **AHA MOMENT #4: THE 4 MODULAR LAWS & FERMAT'S LITTLE THEOREM**
For modulo $M = 10^9 + 7$:
1. **Addition:** `(a + b) % M = ((a % M) + (b % M)) % M`
2. **Subtraction:** `(a - b + M) % M` (Add M to prevent negative result in Java!).
3. **Multiplication:** `((long)(a % M) * (b % M)) % M`
4. **Modular Inverse (Fermat):** $a^{-1} \equiv a^{M-2} \pmod M$ for prime M.

📊 MODULAR ARITHMETIC IMPLEMENTATION MATRIX

Operation	Java Safe Expression	Why Added Safeguard?
Addition	`(a % M + b % M) % M`	Prevents intermediate overflow
Subtraction	`(a % M - b % M + M) % M`	Adds M to avoid negative remainder!
Multiplication	`((long) a * b) % M`	Casts to `long` before multiplying
Division	`(a * modInverse(b, M)) % M`	Uses Fermat's Little Theorem (b^{M-2})

⚠️ **JAVA MODULO BUG:** Java's `%` operator is a remainder, not modulo (`-5 % 3 = -2`). Always add M: `(val % M + M) % M`!

TOPIC 18.4: FAST EXPONENTIATION (BINARY EXP)

💡 **AHA MOMENT #5: BINARY EXPONENTIATION IN $O(\log N)$ TIME**

To compute x^n:

- If n is even ➔ $x^n = (x^2)^{n/2}$
- If n is odd ➔ $x^n = x \times (x^2)^{(n-1)/2}$

Square the base $x = x \times x$ and halve the exponent $n = n / 2$ at each step!

🛠️ POW(X, N) (LC 50) ITERATIVE $O(\LOG N)$ IMPLEMENTATION

```
public double myPow(double x, int n) {
    long N = n; // Protect against Integer.MIN_VALUE overflow when negating!
    if (N < 0) {
        x = 1 / x;
        N = -N;
    }
    double res = 1.0;
    double currentProduct = x;
    while (N > 0) {
        if ((N & 1) == 1) res *= currentProduct; // Odd power
        currentProduct *= currentProduct;        // Square base
        N >>= 1;                                 // Halve exponent
    }
    return res;
}
```

📌 **KEY TAKEAWAY:** `long N = n` prevents overflow when `n = -2147483648` (`Integer.MIN_VALUE`), because `-Integer.MIN_VALUE` overflows 32-bit `int`!

📐 TOPIC 18.5: COMBINATORICS & PASCAL'S TRIANGLE

💡 **AHA MOMENT #6: PASCAL'S TRIANGLE RECURRENCE**

Combination formula: $C(n, k) = \binom{n}{k} = \frac{n!}{k!(n-k)!}$.

Pascal's Triangle Recurrence avoids factorials:

`C(n, k) = C(n - 1, k - 1) + C(n - 1, k)`

Each cell is the sum of the two cells directly above it!

🛠️ PASCAL'S TRIANGLE DYNAMIC PROGRAMMING

```
public List<List<Integer>> generatePascal(int numRows) {
    List<List<Integer>> res = new ArrayList<>();
    for (int i = 0; i < numRows; i++) {
        List<Integer> row = new ArrayList<>();
        for (int j = 0; j <= i; j++) {
            if (j == 0 || j == i) row.add(1);
            else row.add(res.get(i - 1).get(j - 1) + res.get(i - 1).get(j));
        }
        res.add(row);
    }
    return res;
}
```

📌 **KEY TAKEAWAY:** Pascal's Triangle computes combinations without huge factorial multiplications that overflow 64-bit `long`!

🎯 TOPIC 18.6: GEOMETRY & CROSS PRODUCT SLOPE CHECK

💡 **AHA MOMENT #7: COLLINEAR POINT CHECK WITHOUT DIVISION**

Three points (x_1, y_1), (x_2, y_2), and (x_3, y_3) lie on the same straight line IF AND ONLY IF their slopes are equal:
$$\frac{y_2 - y_1}{x_2 - x_1} = \frac{y_3 - y_1}{x_3 - x_1} \implies (y_2 - y_1)(x_3 - x_1) = (y_3 - y_1)(x_2 - x_1)$$
Cross-multiplication avoids `double` division precision errors and division-by-zero (`dy / 0`) vertical line crashes!

🛠️ COLLINEAR POINT CROSS PRODUCT CHECK

```
public boolean isCollinear(int[] p1, int[] p2, int[] p3) {
    long dy1 = p2[1] - p1[1];
    long dx1 = p2[0] - p1[0];
    long dy2 = p3[1] - p1[1];
    long dx2 = p3[0] - p1[0];
    return dy1 * dx2 == dy2 * dx1; // Cross product check (No division!)
}
```

📌 **KEY TAKEAWAY:** Always cross-multiply slope differences to perform exact integer geometric checks!

📜 TOPIC 18.7 & 1-PAGE PRINTABLE MATH CHEAT SHEET

🌳 MATH INTERVIEW DECISION TREE (MERMAID)

📜 PRINTABLE MATH CHEAT SHEET

Algorithm / Formula	Code Snippet	Complexity
Euclidean GCD	`gcd(a, b) = b == 0 ? a : gcd(b, a % b)`	$O(\log(\min(a, b)))$ Time
LCM Formula	`lcm(a, b) = (a / gcd(a, b)) * b`	$O(\log(\min(a, b)))$ Time
Sieve of Eratosthenes	`isPrime[i * p] = false` starting `p * p`	$O(N \log \log N)$ Time, $O(N)$ Space
Binary Exponentiation	`res *= x; x *= x; n >>= 1;`	$O(\log N)$ Time, $O(1)$ Space
Modular Subtraction	`(a - b + M) % M`	$O(1)$ Time
Collinear Points	`dy1 * dx2 == dy2 * dx1`	$O(1)$ Time

🗣️ **FAANG COACH TIP:** Print this page and review it 10 minutes before your technical interview!

MATH TEMPLATE LIBRARY (GCD, LCM & SIEVE)

1. GCD & LCM TEMPLATE

```
long gcd(long a, long b) { return b == 0 ? a : gcd(b, a % b); }
long lcm(long a, long b) {
    if (a == 0 || b == 0) return 0;
    return (a / gcd(a, b)) * b;
}
```

2. SIEVE OF ERATOSTHENES TEMPLATE

```
boolean[] sieve(int n) {
    boolean[] isPrime = new boolean[n + 1];
    Arrays.fill(isPrime, true);
    if (n >= 0) isPrime[0] = false;
    if (n >= 1) isPrime[1] = false;
    for (int p = 2; p * p <= n; p++) {
        if (isPrime[p]) {
            for (int i = p * p; i <= n; i += p) isPrime[i] = false;
        }
    }
    return isPrime;
}
```

📌 **REUSABLE CODE:** Copy-paste these templates directly into your interview whiteboard or IDE!

💻 MATH TEMPLATE LIBRARY (FAST POW & PRIME FACTORIZATION)

3. FAST EXPONENTIATION TEMPLATE

```
double pow(double x, long n) {
    if (n < 0) { x = 1 / x; n = -n; }
    double res = 1.0;
    while (n > 0) {
        if ((n & 1) == 1) res *= x;
        x *= x; n >>= 1;
    }
    return res;
}
```

4. PRIME FACTORIZATION TEMPLATE

```
List<Integer> primeFactors(int n) {
    List<Integer> factors = new ArrayList<>();
    for (int d = 2; d * d <= n; d++) {
        while (n % d == 0) {
            factors.add(d); n /= d;
        }
    }
    if (n > 1) factors.add(n);
    return factors;
}
```

📌 **KEY TAKEAWAY:** Prime factorization checks up to $\sqrt{N}$ in $O(\sqrt{N})$ time!

EASY NEETCODE: COUNT PRIMES & HAPPY NUMBER

1. LEETCODE 204 — COUNT PRIMES

Easy ★★★★★ **Sieve of Eratosthenes** Amazon Apple

Problem:

Count the number of prime numbers strictly less than a non-negative number n.

```
public int countPrimes(int n) {
    if (n ⩽ 2) return 0;
    boolean[] isPrime = new boolean[n];
    Arrays.fill(isPrime, true);
    isPrime[0] = isPrime[1] = false;
    for (int p = 2; p * p < n; p++) {
        if (isPrime[p]) {
            for (int i = p * p; i < n; i += p) isPrime[i] = false;
        }
    }
    int count = 0;
    for (boolean b : isPrime) if (b) count++;
    return count;
}
```

2. LEETCODE 202 — HAPPY NUMBER

Easy ★★★★★ **Cycle Detection (Floyd Fast & Slow)** Amazon Meta

Problem:

Determine if number reaches 1 by repeatedly replacing it with sum of squares of digits.

```
public boolean isHappy(int n) {
    int slow = n, fast = getNext(n);
    while (fast ≠ 1 && slow ≠ fast) {
        slow = getNext(slow);
        fast = getNext(getNext(fast));
    }
    return fast == 1;
}
private int getNext(int n) {
    int sum = 0;
    while (n > 0) {
        int d = n % 10; sum += d * d; n /= 10;
    }
    return sum;
}
```

AHA MOMENT (LC 202): Using Floyd's Fast & Slow pointers detects digit-sum cycles in $O(1)$ extra space!

3. LEETCODE 50 — POW(X, N)

Medium ★★★★★ **Binary Exponentiation** Amazon Google

Problem:

Calculate x^n in $O(\log N)$ time.

```
public double myPow(double x, int n) {
    long N = n;
    if (N < 0) { x = 1 / x; N = -N; }
    double res = 1.0, currentProduct = x;
    while (N > 0) {
        if ((N & 1) == 1) res *= currentProduct;
        currentProduct *= currentProduct;
        N >>= 1;
    }
    return res;
}
```

4. LEETCODE 66 — PLUS ONE

Easy ★★★★★ **Array Carry Propagate** Amazon Google

Problem:

Increment large integer represented as array of digits by 1.

```
public int[] plusOne(int[] digits) {
    for (int i = digits.length - 1; i >= 0; i--) {
        if (digits[i] < 9) {
            digits[i]++; return digits; // No carry required!
        }
        digits[i] = 0; // Carry over
    }
    int[] res = new int[digits.length + 1];
    res[0] = 1; // e.g. 999 → 1000
    return res;
}
```

💡 **AHA MOMENT (LC 66):** If `digits[i] < 9`, incrementing ends immediately without creating a new array!

5. LEETCODE 43 — MULTIPLY STRINGS

Medium ★★★★★ **Digit-by-Digit Array Multiplication** Amazon Meta

Problem:

Multiply two non-negative integers represented as strings without `BigInteger`.

```
public String multiply(String num1, String num2) {
    if (num1.equals("0") || num2.equals("0")) return "0";
    int m = num1.length(), n = num2.length();
    int[] pos = new int[m + n];
    for (int i = m - 1; i ≥ 0; i--) {
        for (int j = n - 1; j ≥ 0; j--) {
            int mul = (num1.charAt(i) - '0') * (num2.charAt(j) - '0');
            int sum = mul + pos[i + j + 1];
            pos[i + j + 1] = sum % 10;
            pos[i + j] += sum / 10; // Carry
        }
    }
    StringBuilder sb = new StringBuilder();
    for (int p : pos) if (!(sb.length() == 0 && p == 0)) sb.append(p);
    return sb.toString();
}
```

6. LEETCODE 396 — ROTATE FUNCTION

Medium ★★★★★ **Mathematical State Recurrence** Amazon

Problem:

Maximum value of $F(k) = \sum i \times A[i]$ after rotating array k times.

```
public int maxRotateFunction(int[] nums) {
    int sum = 0, f = 0, n = nums.length;
    for (int i = 0; i < n; i++) {
        sum += nums[i]; f += i * nums[i];
    }
    int maxF = f;
    for (int i = n - 1; i ≥ 1; i--) {
        f = f + sum - n * nums[i]; // Recurrence formula!
        maxF = Math.max(maxF, f);
    }
    return maxF;
}
```

💡 **AHA MOMENT (LC 396):** Recurrence formula `F(k) = F(k-1) + sum - n * nums[n-k]` computes rotation score in $O(1)$ time instead of $O(N)$!

7. LEETCODE 227 — BASIC CALCULATOR II

Medium ★★★★★ Operator Precedence Parsing Amazon Meta

Problem:

Evaluate string expression containing `+`, `-`, `*`, `/` and non-negative integers.

```
public int calculate(String s) {
    int len = s.length(), curr = 0, res = 0, last = 0;
    char op = '+';
    for (int i = 0; i < len; i++) {
        char c = s.charAt(i);
        if (Character.isDigit(c)) curr = curr * 10 + (c - '0');
        if ((!Character.isDigit(c) && c != ' ') || i == len - 1) {
            if (op == '+') { res += last; last = curr; }
            else if (op == '-') { res += last; last = -curr; }
            else if (op == '*') { last = last * curr; }
            else if (op == '/') { last = last / curr; }
            op = c; curr = 0;
        }
    }
    return res + last;
}
```

8. LEETCODE 149 — MAX POINTS ON A LINE

Hard ★★★★★ GCD Reduced Slope Frequency Map Amazon Google

Problem:

Maximum number of points that lie on the same 2D straight line.

```
public int maxPoints(int[][] points) {
    int n = points.length; if (n <= 2) return n;
    int maxPts = 0;
    for (int i = 0; i < n; i++) {
        Map<String, Integer> map = new HashMap<>(); int maxGroup = 0;
        for (int j = i + 1; j < n; j++) {
            int dy = points[j][1] - points[i][1], dx = points[j][0] -
points[i][0];
            int g = gcd(dy, dx);
            String key = (dy / g) + "/" + (dx / g); // Reduced slope
key!
            map.put(key, map.getOrDefault(key, 0) + 1);
            maxGroup = Math.max(maxGroup, map.get(key));
        }
        maxPts = Math.max(maxPts, maxGroup + 1);
    }
    return maxPts;
}
private int gcd(int a, int b) { return b == 0 ? a : gcd(b, a % b); }
```

FAANG COACH TIP: `dy / gcd(dy, dx)` and `dx / gcd(dy, dx)` reduces slopes into exact integer string keys without float precision bugs!

ADVANCED GEOMETRY: DETECT SQUARES & SPIRAL MATRIX

9. LEETCODE 2013 — DETECT SQUARES

Medium ★★★★★ **Diagonal Corner Search** Amazon Google

Problem:

Count number of ways to form axis-aligned squares with query point.

```
class DetectSquares {
    private List<int[]> pts = new ArrayList<>();
    private int[][] counts = new int[1001][1001];
    public void add(int[] point) {
        pts.add(point); counts[point[0]][point[1]]++;
    }
    public int count(int[] point) {
        int px = point[0], py = point[1], total = 0;
        for (int[] p : pts) {
            int x = p[0], y = p[1];
            if (Math.abs(px - x) != Math.abs(py - y) || px == x || py ==
y) continue;
            total += counts[px][y] * counts[x][py]; // Product of
opposite corners
        }
        return total;
    }
}
```

10. LEETCODE 54 — SPIRAL MATRIX

Medium ★★★★★ **Boundary Shrinking Simulation** Amazon Apple

Problem:

Return all elements of $m \times n$ matrix in spiral order.

```
public List<Integer> spiralOrder(int[][] matrix) {
    List<Integer> res = new ArrayList<>();
    int top = 0, bottom = matrix.length - 1, left = 0, right =
matrix[0].length - 1;
    while (top <= bottom && left <= right) {
        for (int j = left; j <= right; j++) res.add(matrix[top][j]);
top++;
        for (int i = top; i <= bottom; i++) res.add(matrix[i][right]);
right--;
        if (top <= bottom) {
            for (int j = right; j >= left; j--) res.add(matrix[bottom]
[j]); bottom--;
        }
        if (left <= right) {
            for (int i = bottom; i >= top; i--) res.add(matrix[i]
[left]); left++;
        }
    }
    return res;
}
```

📌 **KEY TAKEAWAY:** LC 2013 checks diagonal points `|px - x| == |py - y|` to locate potential opposite corners in $O(N)$ time!

💎 IF YOU REMEMBER ONLY ONE PAGE... MASTER REVISION SHEET

⭐ THE 4 GOLDEN LAWS OF MATHEMATICS IN CODING INTERVIEWS

1. **Euclidean Law:** `gcd(a, b) = gcd(b, a % b)` computes Greatest Common Divisor in $O(\log(\min(a, b)))$ time.
2. **LCM Safety Law:** Always compute `(a / gcd(a, b)) * b` (divide FIRST!) to avoid integer overflow.
3. **Fast Exponentiation Law:** Binary exponentiation halving exponent N computes x^N in $O(\log N)$ time.
4. **Cross Product Law:** Use `dy1 * dx2 == dy2 * dx1` for slope checks without division precision errors!

⛔ COMMON MATH BUGS & PITFALLS

- **Negating `Integer.MIN_VALUE`:** `-Integer.MIN_VALUE` STILL equals `-2147483648`! Cast to `long` before negating.
- **Double Precision Comparison:** Never use `double1 == double2`. Use `Math.abs(d1 - d2) < 1e-9`.

📊 NEETCODE & BLIND 75 CONFIDENCE TRACKER

🟢 Easy LC 204 Count Primes	🟢 Easy LC 202 Happy Number	🟢 Easy LC 66 Plus One
🟠 Med LC 50 Pow(x, n)	🟠 Med LC 43 Multiply Strings	🟠 Med LC 396 Rotate Function
🟠 Med LC 227 Basic Calculator II	🟠 Med LC 2013 Detect Squares	🟠 Med LC 54 Spiral Matrix
🔴 Hard LC 149 Max Points on Line		

👉 **Next Master Topic: Topic 19 → Advanced Data Structures Masterclass**

🎉 **TOPIC 18 MATHEMATICS MASTERCLASS COMPLETE!** Turn the page to **Topic 19: Advanced Data Structures Masterclass!**